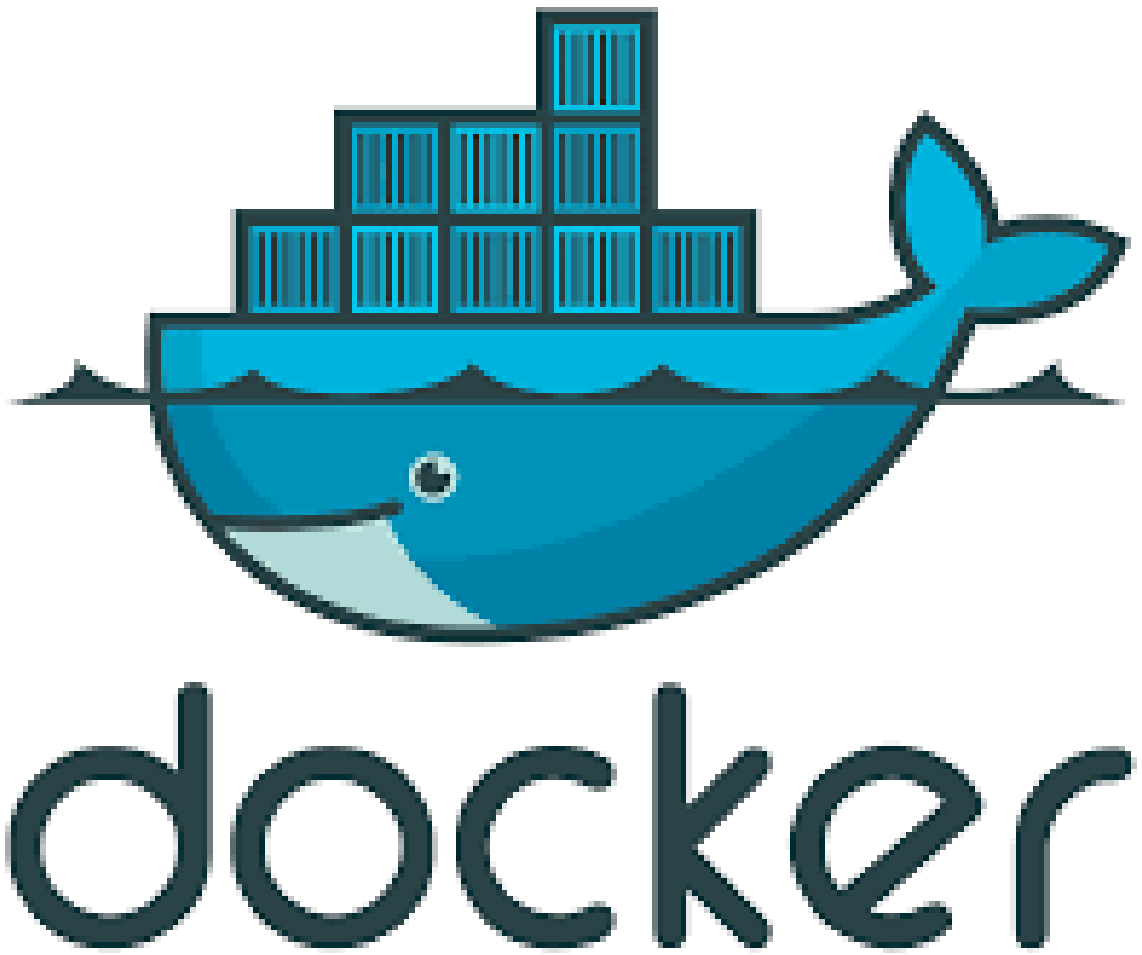
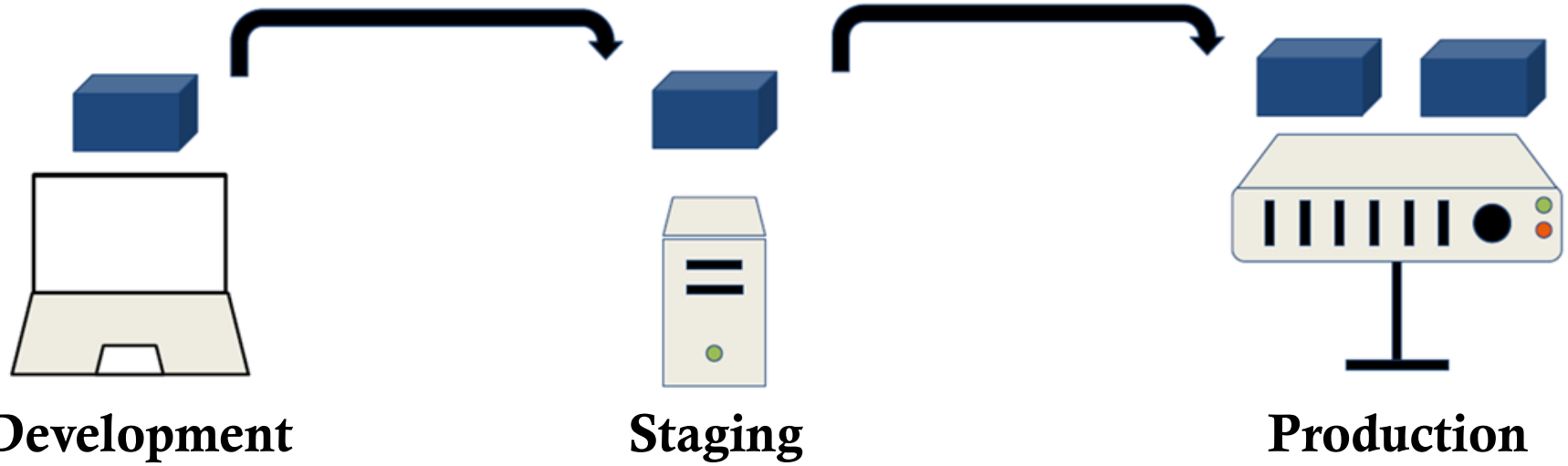

docker

INTRODUCTION TO DOCKER

Environment Management

- Have **multiple environments** on the same host machine.
- These environments are **isolated**.
- Maintain the **same environment throughout the entire production pipeline** (standardization concept).
- Be able to **replicate the same environment**



Virtualization and containerization

Virtual machines

A **virtual machine (VM)** hosts a complete environment including:

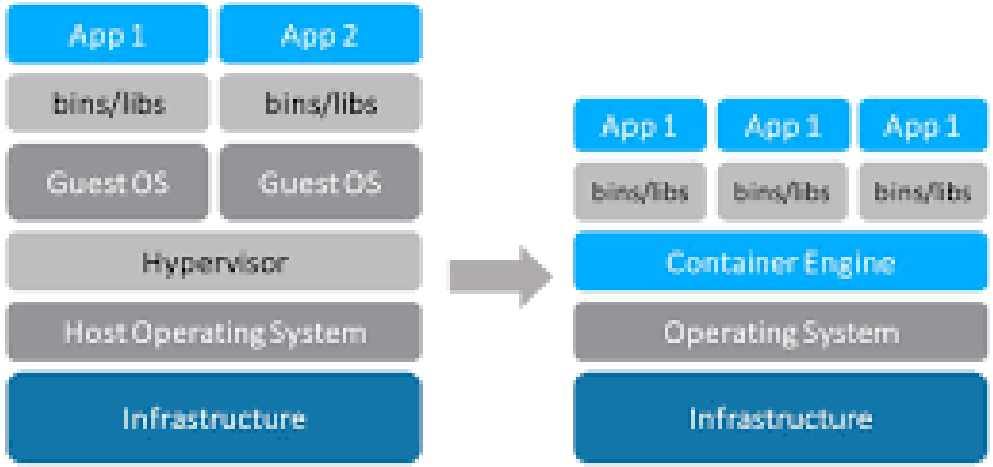
- its own **operating system**
- its own **libraries and binaries**
- its own **applications**

Each virtual machine is managed by a **hypervisor**.

Containers

A **container** does not include a full operating system. Instead, containers:

- share the **host machine kernel**
- package only the **libraries, binaries, and dependencies** required by the application
- run through a container runtime such as **Docker Engine**



Virtualization and containerization

Virtual machines

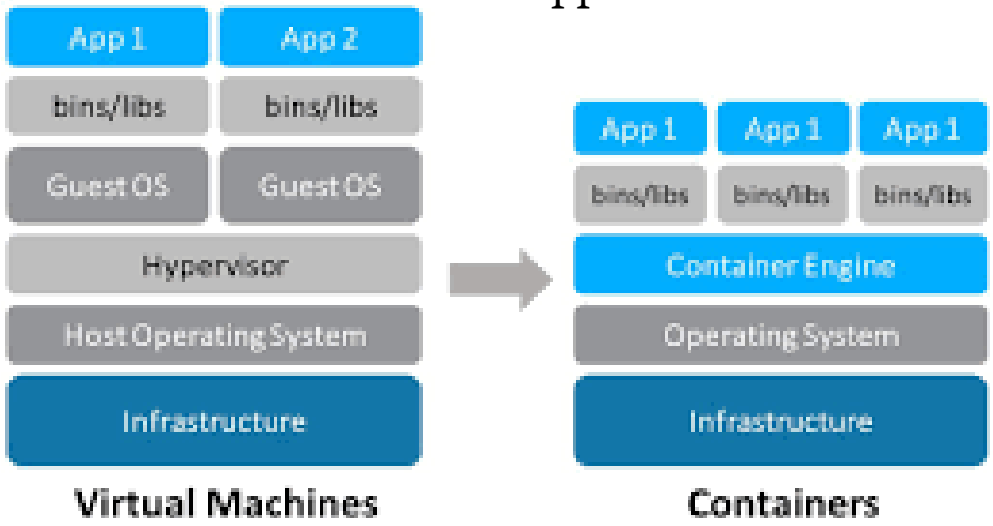
Key idea: each VM contains a full guest OS, which makes virtualization powerful but generally heavier in terms of resources and startup time.

Containers

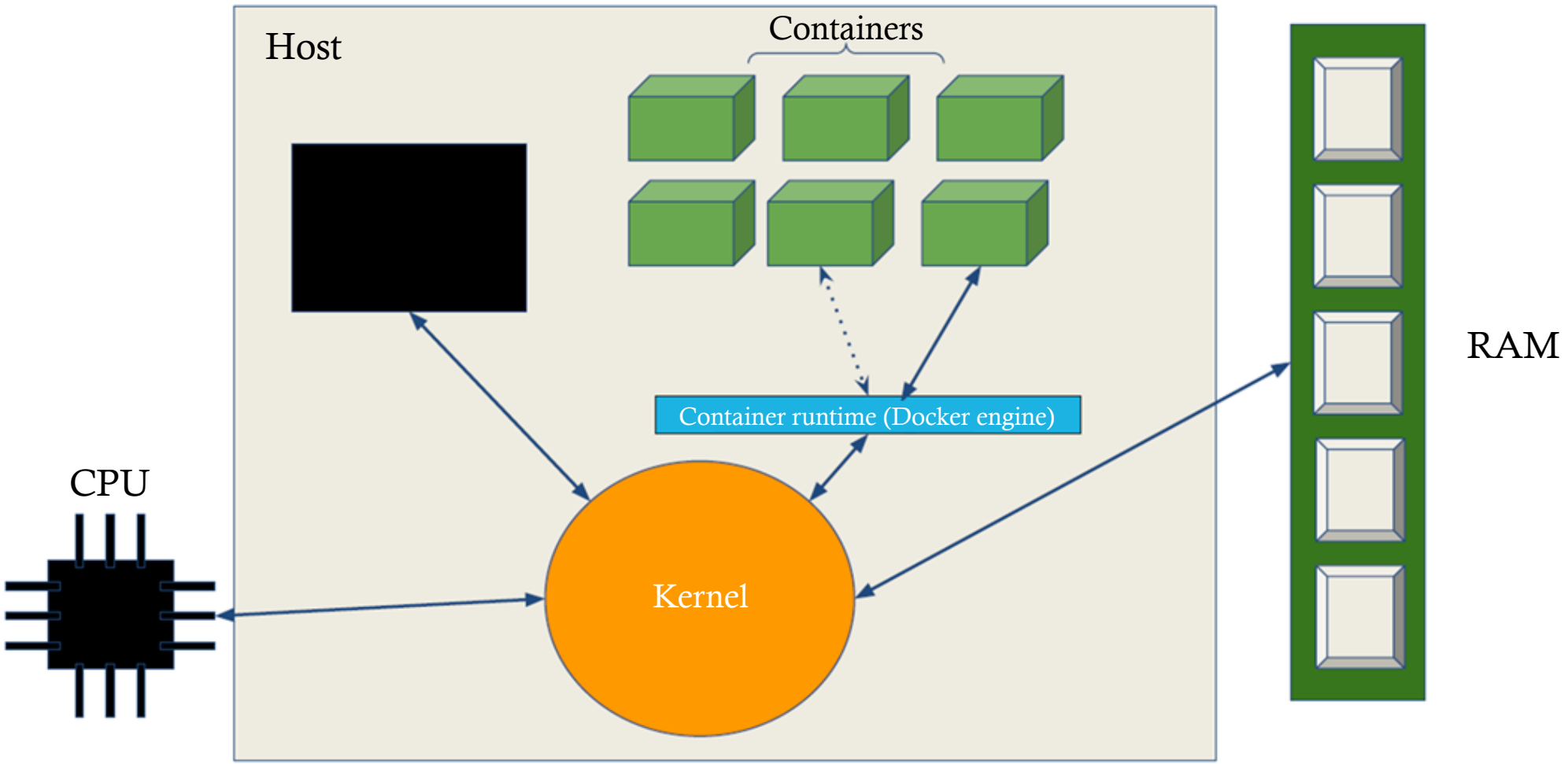
Key idea: containers are usually:

- lighter
- faster to start
- easier to replicate and deploy consistently

because they avoid duplicating a complete OS for each application.

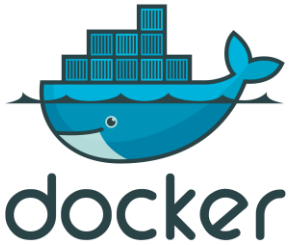


INTRODUCTION TO DOCKER



INTRODUCTION TO DOCKER

What is Docker?



Docker is a container platform that makes applications portable by packaging them with their dependencies and running them as isolated processes on a shared operating system.

Docker on Linux/Windows

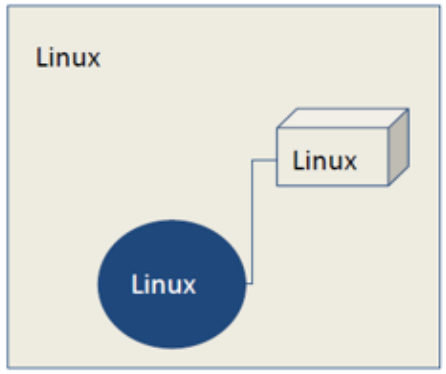
Containers rely on the host operating system's kernel to run.

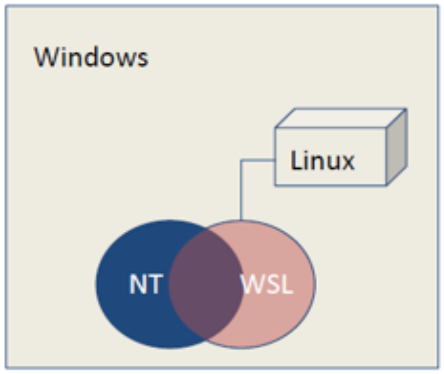
A Linux container shares the host's Linux kernel.

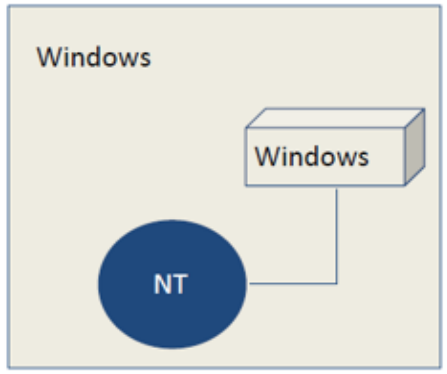
So, how does Docker work on Windows?

Docker can run two different types of containers on Windows:

- A Windows container packages a Windows user-space environment and runs directly on the Windows NT kernel.
- Most developers use Linux containers on Windows through Windows Subsystem for Linux (WSL 2).







INTRODUCTION TO DOCKER

A brief history

2008-2012

Linux technologies such as cgroups and namespaces make operating-system-level virtualization possible, enabling isolated environments without the overhead of full virtual machines.

2013

Docker is introduced by Solomon Hykes at the Python Conference (PyCon). Originally an internal project at dotCloud, Docker simplifies container creation and deployment.

2014

Docker gains rapid adoption and releases version 1.0. The company also launches Docker Hub, making it easy to share container images.

2015-2017

Container orchestration becomes important as applications scale. Docker competes with and influences orchestration systems such as Docker Swarm and Kubernetes.

2017

Docker donates its container runtime technology to the Cloud Native Computing Foundation through projects such as containerd, helping standardize the container ecosystem.

Who is it intended for?

Developers

Using Docker, developers can package each application together with its runtime, dependencies, and configuration.

Different applications may require different technology stacks. For example, one application may use PHP 7.3 and Apache, while another uses PHP 8.1 and Nginx. Docker allows these applications to run in isolated containers without conflicts.

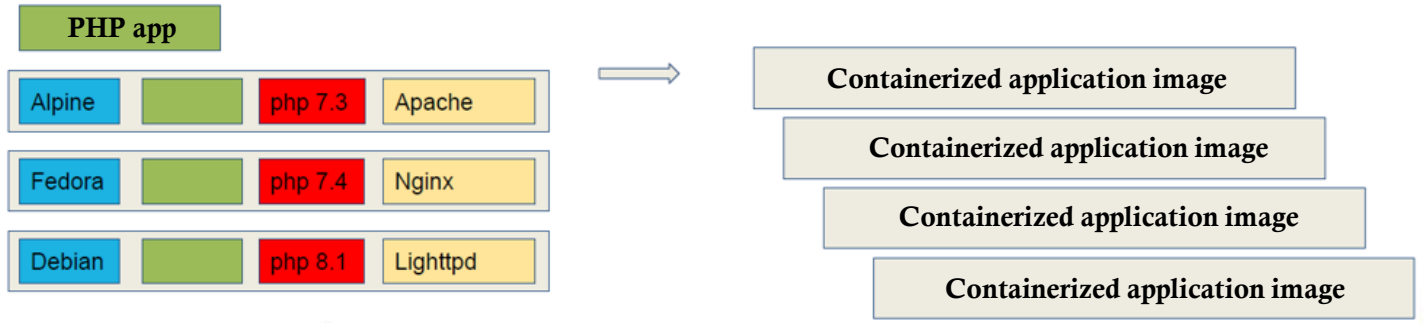
It also enables developers to test the same application across multiple environments, ensuring consistent behavior from development to production.

System admins / Operations

Once the application has been packaged as a Docker image, it can be deployed repeatedly across development, testing, and production environments.

Administrators no longer need to manually configure each server. The same image can be deployed anywhere, ensuring consistency, simplifying updates, and making applications easier to monitor and maintain.

Example: A PHP application can be containerized with a specific Linux distribution, PHP version, and web server, creating a portable and reproducible environment.



Benefits

For system admins

Isolation

Containers provide process, filesystem, and network isolation from the host system and from each other.

Each container runs in a controlled environment, reducing the impact of a compromised application on the host or other containers.

However, containers share the host operating system kernel, so isolation is not as strong as with virtual machines.

Multiple versions

Containers make it possible to run different applications, or different versions of the same software, on the same host without conflicts.

For example:

- one application may require PHP 5.x,
- another may require PHP 7.x,
- different services may also depend on different versions of databases such as MySQL.

Each container includes its own dependencies and runtime environment.

Benefits

For system admins

Cost efficiency

A single physical or virtual machine can be divided into multiple lightweight containers.

This allows better utilization of CPU and memory resources compared to running separate virtual machines for each application.

Resource limits (CPU, memory, etc.) can be enforced using kernel-level controls such as cgroups.

Scalability

Containers can be easily replicated to handle changes in workload.

With container orchestration platforms such as Kubernetes, applications can be automatically scaled up or down, load-balanced, and restarted if they fail.

Benefits

For developers

One of the classic problems in software development is: ***"It works on my machine."***

Docker solves this by standardizing environments. Instead of each developer (or even the client) running slightly different setups, everyone works from the same container image. This ensures that the application behaves consistently across all stages (development, testing, and production).

Benefits

For developers

CI/CD improvements

Docker…:

- … enables automated testing in clean and reproducible environments. Tests can be executed on isolated containers in CI pipelines, ensuring that they run in an environment as close as possible to production. This avoids issues caused by polluted local environments and reduces inconsistencies between machines.
- … simplifies deployment. Since your application is packaged inside a Docker image, there is no need to manually configure the production server with specific dependencies or runtime versions. Whether you deploy to a physical server, a VM, or a cloud platform, you simply run the container. Everything needed for the application is already included in the image, which makes deployments more predictable and portable.

Benefits

For developers

Working with multiple versions

- Docker is also very useful when dealing with multiple versions of software or frameworks.
- For example, in a long-lived project, older versions of your application may still be in production while new versions are being developed. Docker allows you to maintain and test fixes on legacy versions without interfering with your current development environment.
- It is also helpful when preparing for future upgrades. If a new version of a framework is released, you can safely test it in an isolated container environment without affecting your main system. This makes it easier to experiment with beta versions or migrations while keeping your local setup stable.

INTRODUCTION TO DOCKER

Terminology

- **Container**: a lightweight, isolated runtime environment that contains your application and everything it needs to run (code, dependencies, libraries). You can think of it as a "mini computer" running your app.
- **Docker image**: a blueprint used to create containers. It is a read-only template that includes your application code, runtime, system tools, and dependencies. When you run an image, it becomes a container.
- **Dockerfile**: a text file where you define how to build an image. It contains step-by-step instructions like which base image to use, what dependencies to install, and what command should start your app.
- **Docker daemon**: the background service that actually builds, runs, and manages containers. When you run Docker commands, you are communicating with this daemon.
- **Docker client**: the command-line tool (docker) used to interact with Docker. It sends instructions to the daemon.
- **Registry**: where images are stored and shared. The most common one is Docker Hub, but companies often use private registries as well.

INTRODUCTION TO DOCKER

Terminology

- **Volume**: a way to persist data outside of a container. Since containers are ephemeral (they can be deleted and recreated), volumes ensure your data is not lost.
- **Network**: in Docker, allows containers to communicate with each other (for example, a backend talking to a database container).
- **Container orchestration systems**: used when you need to manage many containers across multiple machines, handling scaling, deployment, and reliability (ex.: Kubernetes).
- **Stateless containers**: containers that do not keep any data internally between runs. If the container stops or is deleted, nothing important is lost inside it. Every request is independent. You can scale them easily (just spin up more identical containers). Typical examples include web servers, APIs, frontend services, microservices that call external databases. This is the "ideal" model for Docker, because containers are meant to be ephemeral (temporary and replaceable).
- **Stateful containers**: containers that store data locally inside the container. If the container is removed, data can be lost (unless properly handled). Scaling is more complex because each instance may have different data. Typical examples include databases (PostgreSQL, MySQL, Redis) and file storage systems. In Docker, statefulness is usually handled **using volumes (or** external storage systems), so that the data lives outside the container itself.

IMAGE AND CONTAINER MANAGEMENT

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

In this section, try running some of the commands.

You can use an interactive shell and a long running-service.

```
docker run -it ubuntu bash
```

```
docker run nginx
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

docker run

Create and start a container from an image.

```
docker run hello-world
```

image

Guess what these commands do (and try to run some of them):

```
docker run -it ubuntu bash
```

```
docker run -p 8080:80 nginx
```

```
docker run -d nginx
```

```
docker run --name my-app nginx
```

```
docker run -v $(pwd):/app my-app
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

docker run

-i = interactive (keep input open)

-t = terminal (gives you a shell)

-p maps ports. In our example 8080:80, 8080 is your machine, 80 is inside container, so you can open http://localhost:8080.

-d = detached (runs in background)

Container management commands

docker ps

Show all currently running containers on your system.

- CONTAINER ID: unique identifier of the container
- IMAGE: which image it was created from
- COMMAND: command running inside the container
- STATUS: running time or state
- PORTS: exposed ports
- NAMES: container name (auto-generated or user-defined)

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Container management commands

docker ps

`docker ps -a` : show all containers, including stopped, exited and crashed ones.

`docker ps -q` : show only container IDs (useful for scripting, for example stopping all containers).

`docker ps --filter "status=exited"` : filter containers (by status, name, ancestor image).

`docker ps --format "{{.Names}}: {{.Status}}"` : format outputs.

Container management commands

docker logs

Used to view the output (stdout and stderr) of a container.

One of the first commands you will use when troubleshooting a container that is not behaving as expected.

-f = follow: follow logs in real time

--tail = show only the last lines

```
docker logs --tail 50 my-app
```

-t = include timestamps

Usually, logs should be written to the console rather than to files inside the container.

Container management commands

docker stop

Gracefully stop a running container.

Docker accepts either the container name or ID.

```
docker stop my-app
```

```
docker stop 7f3c2a1b9d4e
```

After `docker stop`, the container still exists and can be restarted.

Container management commands

docker stop

What happens internally?

Docker:

1. Sends a SIGTERM signal to the main process inside the container.
2. Waits for the process to shut down cleanly (default: 10 seconds on Linux).
3. If the process is still running, Docker sends SIGKILL to force termination.

This gives the application a chance to finish ongoing requests, close database connections, flush logs, save state…

Container management commands

docker stop

Stop multiple containers:

```
docker stop container1 container2 container3
```

```
docker stop $(docker ps -q)
```

Container management commands

docker start

Start an existing stopped container.

```
docker start container
```

Unlike docker run, which creates a new container and starts it, docker start simply starts a container that already exists.

Container management commands

docker restart

Equivalent to:

```
docker stop my-app
docker start my-app
```

Container management commands

docker rm

Remove one or more containers from your system.

With the basic command, you cannot remove a running container.

```
docker stop my-app
docker rm my-app
```

or

```
docker rm -f my-app
```

Container management commands

docker rm

Very common cleanup command:

```
docker container prune
```

or

```
docker rm $(docker ps -a -q)
```

Image management

docker images

List the Docker images stored on your local machine.

```
REPOSITORY          TAG         IMAGE ID       CREATED          SIZE
ubuntu              24.04       abc123def456   2 weeks ago      77MB
nginx               latest      789xyz654abc   1 month ago      192MB
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker images

Show all images, including intermediate layers: `docker images -a`

Show only image IDs: `docker images -q`

Filter by repository name: `docker images nginx`

Display images with a custom format:

```
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
```

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker pull

Download a Docker image from a registry (by default, Docker Hub) to your local machine.

```
docker pull <image>:<tag>
```

What happens when you run it?

Docker checks if the image already exists locally. If not (or if a newer version is requested), it downloads image layers and metadata, then it stores it in your local image cache

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker pull

Download a Docker image from a registry (by default, Docker Hub) to your local machine.

```
docker pull <image>:<tag>
```

Always pull newer version (ignore cache): `docker pull --pull`

Quiet mode (less output): `docker pull -q nginx`

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker build

Create a Docker image from a Dockerfile. It takes the instructions in a Dockerfile and turns them into a runnable image.

```
docker build -t <image-name>:<tag> <path>
```

`-t` names (tags) the image
`<path>` directory containing the Dockerfile (often `.`)

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker build

No cache (force rebuild everything): `docker build --no-cache -t myapp .`

Specify a different Dockerfile: `docker build -f Dockerfile.prod -t myapp .`

Build from a Git repo: `docker build https://github.com/user/repo.git`

Common flow

```
docker build -t my-app .
docker run -p 3000:3000 my-app
```

Image management

docker tag

`docker tag` is a command that adds a name (tag) to an existing Docker image. It does not create a new image, it just creates another reference to the same image (think of it like giving the same photo multiple filenames).

```
docker tag <source-image>:<tag> <target-image>:<tag>
```

Image management

docker tag

Two common examples of this are:

- Tagging your image as the latest version: `docker tag my-app:2.0 my-app:latest`
- Tagging it for a registry (for example, Docker Hub): `docker tag myapp:latest username/myapp:1.0`

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker push

Upload a Docker image from your local machine to a remote registry (Docker Hub, GitHub Container Registry, AWS ECR…).

```
docker push <image>:<tag>
```

Make sure to tag your image correctly first!

What docker push actually does:

When you run it, Docker:

1. Checks which layers already exist in the registry
2. Uploads only missing layers (efficient, incremental upload)
3. Publishes metadata for the tag
4. Makes the image available remotely

IMAGE AND CONTAINER MANAGEMENT TO DOCKER

Image management

docker push

The image name must match a registry format:

Registry	Format
Docker Hub	`username/repo:tag`
GHCR	`ghcr.io/user/repo:tag`
AWS ECR	`account-id.dkr.ecr.region.amazonaws.com/repo:tag`

Image management

docker push

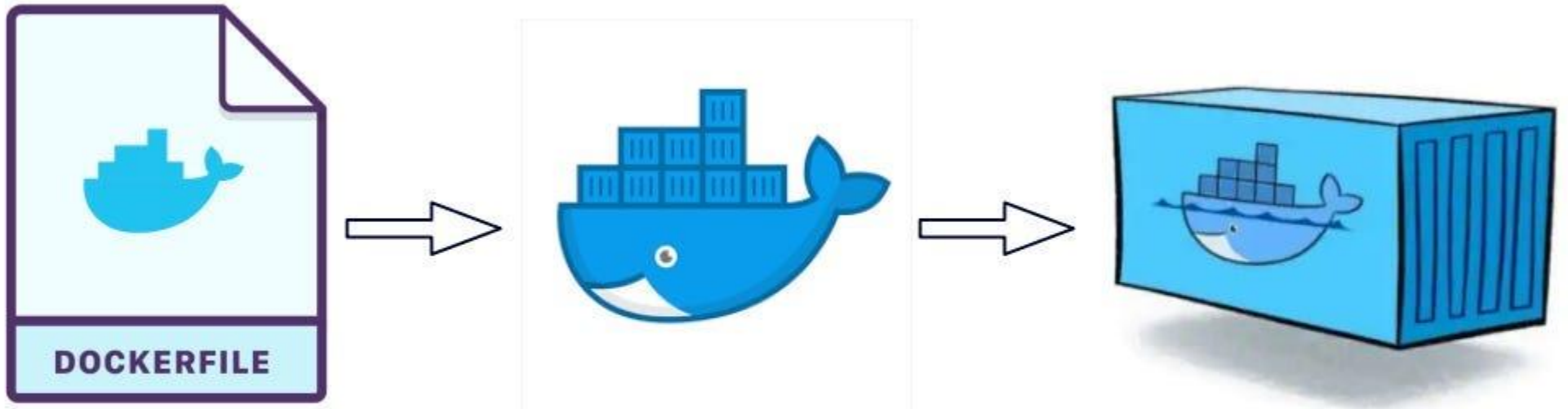
You can also push a floating tag (usually latest):

```
docker tag my-app:2.0 username/my-app:latest
docker push username/my-app:latest
```

Be careful because of course this will overwrite what “latest” means in the registry.

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE



BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

Let's have a look at some Dockerfile and docker-compose.yml files.

WHAT IS TRUE ABOUT DOCKER IMAGES AND CONTAINERS

- ❑ A CONTAINER IS A TEMPLATE USED TO BUILD IMAGES
- ❑ AN IMAGE IS A RUNNING INSTANCE OF A CONTAINER
- ❑ AN IMAGE IS A READ-ONLY TEMPLATE USED TO CREATE CONTAINERS
- ❑ THEY ARE IDENTICAL CONCEPTS

WHAT IS TRUE ABOUT DOCKER IMAGES AND CONTAINERS

- ☐ A CONTAINER IS A TEMPLATE USED TO BUILD IMAGES
- ☐ AN IMAGE IS A RUNNING INSTANCE OF A CONTAINER
- ☑ AN IMAGE IS A READ-ONLY TEMPLATE USED TO CREATE CONTAINERS
- ☐ THEY ARE IDENTICAL CONCEPTS

WHAT HAPPENS IF YOU REMOVE A CONTAINER WITHOUT USING A VOLUME FOR STORAGE?

- THE IMAGE IS DELETED BUT DATA REMAINS
- THE CONTAINER RESTARTS AUTOMATICALLY
- ALL DATA STORED INSIDE THE CONTAINER IS LOST
- DOCKER BACKS UP THE DATA AUTOMATICALLY

WHAT HAPPENS IF YOU REMOVE A CONTAINER WITHOUT USING A VOLUME FOR STORAGE?

- ❑ THE IMAGE IS DELETED BUT DATA REMAINS
- ❑ THE CONTAINER RESTARTS AUTOMATICALLY
- ☑ ALL DATA STORED INSIDE THE CONTAINER IS LOST
- ❑ DOCKER BACKS UP THE DATA AUTOMATICALLY

WHICH STATEMENT BEST DESCRIBES THE PURPOSE OF DOCKER NETWORKING?

- ❑ IT ENCRYPTS CONTAINER IMAGES
- ❑ IT ALLOWS CONTAINERS TO COMMUNICATE WITH EACH OTHER AND EXTERNAL SYSTEMS
- ❑ IT REPLACES THE NEED FOR ENVIRONMENT VARIABLES
- ❑ IT IS USED FOR LOGGING CONTAINER OUTPUT

WHICH STATEMENT BEST DESCRIBES THE PURPOSE OF DOCKER NETWORKING?

- ❑ IT ENCRYPTS CONTAINER IMAGES
- ☑ IT ALLOWS CONTAINERS TO COMMUNICATE WITH EACH OTHER AND EXTERNAL SYSTEMS
- ❑ IT REPLACES THE NEED FOR ENVIRONMENT VARIABLES
- ❑ IT IS USED FOR LOGGING CONTAINER OUTPUT

WHY ARE CONTAINERS CONSIDERED MORE LIGHTWEIGHT THAN VIRTUAL MACHINES?

- THEY DO NOT USE ANY CPU RESOURCES
- THEY SHARE THE HOST OPERATING SYSTEM KERNEL
- THEY RUN WITHOUT ANY OPERATING SYSTEM
- THEY RUN FASTER THAN NATIVE APPLICATIONS

WHY ARE CONTAINERS CONSIDERED MORE LIGHTWEIGHT THAN VIRTUAL MACHINES?

- ☐ THEY DO NOT USE ANY CPU RESOURCES
- ☑ THEY SHARE THE HOST OPERATING SYSTEM KERNEL
- ☐ THEY RUN WITHOUT ANY OPERATING SYSTEM
- ☐ THEY RUN FASTER THAN NATIVE APPLICATIONS

WHAT IS THE MAIN ADVANTAGE OF USING ENVIRONMENT VARIABLES IN DOCKER?

- ❑ THEY ALLOW MODIFICATION OF THE IMAGE WITHOUT REBUILDING IT
- ❑ THEY PERMANENTLY STORE DATA INSIDE CONTAINERS
- ❑ THEY REPLACE THE NEED FOR DOCKERFILES
- ❑ THEY INCREASE CONTAINER ISOLATION AT KERNEL LEVEL

WHAT IS THE MAIN ADVANTAGE OF USING ENVIRONMENT VARIABLES IN DOCKER?

- [x] THEY ALLOW MODIFICATION OF THE IMAGE WITHOUT REBUILDING IT
- [ ] THEY PERMANENTLY STORE DATA INSIDE CONTAINERS
- [ ] THEY REPLACE THE NEED FOR DOCKERFILES
- [ ] THEY INCREASE CONTAINER ISOLATION AT KERNEL LEVEL

LAB

Create three to five questions about what we covered during this first training block

Multiple choice questions
4 options, 1 correct answer

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

Dockerfile structure

Docker builds images by reading the instructions from a Dockerfile. A Dockerfile is a text file containing instructions for building your source code.

The Dockerfile instruction syntax is defined by the specification reference in the Dockerfile reference.

Most common instructions:

Instruction	Description
`FROM <image>`	Defines a base for your image.
`RUN <command>`	Executes any commands in a new layer on top of the current image and commits the result. `RUN` also has a shell form for running commands.
`WORKDIR <directory>`	Sets the working directory for any `RUN`, `CMD`, `ENTRYPOINT`, `COPY`, and `ADD` instructions that follow it in the Dockerfile.
`COPY <src> <dest>`	Copies new files or directories from `<src>` and adds them to the filesystem of the container at the path `<dest>`.
`CMD <command>`	Lets you define the default program that is run once you start the container based on this image. Each Dockerfile only has one `CMD`, and only the last `CMD` instance is respected when multiple exist.

Dockerfile - filename

- The standard filename for a Docker build file is `Dockerfile`, with no file extension.
- When you use this default name, you can run the `docker build` command without providing any extra options.
- In some projects, multiple Dockerfiles are used for different purposes. A common naming convention is `<something>.Dockerfile` (for example, `dev.Dockerfile` or `test.Dockerfile`). When using a Dockerfile with a non-default name, specify it with the `--file` (or `-f`) option in the `docker build` command.
- It is however recommended to use the default Dockerfile name for your project's primary Dockerfile.

Layers

The Docker images we introduced consist of layers. Each layer is the result of a build instruction in the Dockerfile. Layers are stacked sequentially, and each one is a delta representing the changes applied to the previous layer.

An example

The following example code shows a small "Hello World" application written in Python, using the Flask framework.

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "Hello World!"
```

In order to ship and deploy this application without Docker Build, you would need to make sure that:

- the required runtime dependencies are installed on the server
- the Python code gets uploaded to the server's filesystem
- the server starts your application, using the necessary parameters

LAB

SETTING UP A DOCKERIZED ENVIRONMENT

An example - syntax parser directive

Let's write our Dockerfile together.

While optional, the first line to add to a Dockerfile is a **# syntax parser directive** if we choose to use one.

This directive instructs the Docker builder what syntax to use when parsing the Dockerfile, and allows older Docker versions with BuildKit enabled to use a specific Dockerfile frontend before starting the build. Parser directives must appear before any other comment, whitespace, or Dockerfile instruction in your Dockerfile, and should be the first line in Dockerfiles.

The recommended syntax is docker/dockerfile:1, which always points to the latest release of the version 1 syntax. BuildKit automatically checks for updates of the syntax before building, making sure you are using the most current version.

Add this first line to your Dockerfile.

```
# syntax=docker/dockerfile:1
```

Parser directives

Along with syntax, there are two other parser directives:

Escape

```
# escape=\
```

```
# escape=`
```

- The escape directive sets the character used to escape characters in a Dockerfile. If not specified, the default escape character is \.
- The escape character is used both to escape characters in a line, and to escape a newline. This allows a Dockerfile instruction to span multiple lines. Note that regardless of whether the escape parser directive is included in a Dockerfile, escaping is not performed in a RUN command, except at the end of a line.
- Setting the escape character to ` is especially useful on Windows, where \ is the directory path separator. ` is consistent with Windows PowerShell.

Parser directives

Along with syntax, there are two other parser directives:

Check

```
# check=skip=<checks|all>
# check=error=<boolean>
```

The **check** directive controls how Docker build checks are handled. By default, Docker runs all available checks and reports any failures as warnings rather than stopping the build.
To disable a specific check, use:

```
#check=skip=<check-name>
```

To specify multiple checks to skip, separate them with a comma:

```
# check=skip=JSONArgsRecommended,StageNameCasing
```

Parser directives

Along with syntax, there are two other parser directives:

Check

By default, builds with failing build checks exit with a zero status code despite warnings. To make the build fail on warnings, set

```
# check=error=true
```

An example – base image

Set your base image to the 22.04 release of Ubuntu. We already know the instruction since it is one of the very common ones we mentioned.

All instructions that follow are executed in this base image. The notation `ubuntu:22.04` follows the `name:tag` standard for naming Docker images. When you build images, you use this notation to name your images.

An example – environment setup

Execute a shell in Ubuntu that updates the APT package index and installs the python3 and python3-pip Python tools in the container.

DID YOUR PREVIOUS LINE EXECUTE A BUILD COMMAND INSIDE THE BASE IMAGE?

- ❑ YES
- ❑ NO

DID YOUR PREVIOUS LINE EXECUTE A BUILD COMMAND INSIDE THE BASE IMAGE?

- [x] YES
- [ ] NO

An example – comments

Comments in Dockerfiles begin with the # symbol. As your Dockerfile evolves, comments can be instrumental to document how your Dockerfile works for any future readers and editors of the file, including your future self.

You might've noticed that comments are denoted using the same symbol as the syntax directive on the first line of the file. The symbol is only interpreted as a directive if the pattern matches a directive and appears at the beginning of the Dockerfile. Otherwise, it's treated as a comment.

Add a comment indicating « install app dependencies » before your last instructions.

An example – another dependency

Not all our dependencies have been installed yet. Let's install Flask.

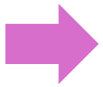
Run the following instruction: `pip install flask==3.0.*`

A prerequisite for this instruction is that pip is installed into the build container. Our first RUN command installed pip, which ensures that we can use the command to install the flask web framework.

An example – copying files

It is time to copy our hello.py file from the local build context into the root directory of our image.

Use a COPY instruction to do so.



A build context is the set of files that you can access in Dockerfile instructions such as COPY and ADD.

After the COPY instruction, the hello.py file is added to the filesystem of the build container.

An example – setting environment variables

If your application uses environment variables, you can set environment variables in your Docker build using the ENV instruction.

Add the following instruction to your Dockerfile:

```
ENV FLASK_APP=hello
```

This sets a Linux environment variable we need later. Flask, the framework used in this example, uses this variable to start the application. Without this, flask wouldn't know where to find our application to be able to run it.

An example – exposed ports

The EXPOSE instruction indicates that the containerized application listens on a specific network port.

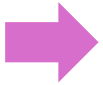
Use the EXPOSE instruction to expose port 8000 in the Dockerfile.

This instruction isn't required, but it is a good practice and helps tools and team members understand what this application is doing.

An example – starting the application

The CMD instruction sets the command that is run when the user starts a container based on this image.

Start the flask development server listening on all addresses on port 8000. Use host 0.0.0.0 to listen on all network interfaces inside the container. You can choose either the "exec form" version or the “shell form” version of CMD.



These two syntaxes are not completely equivalent; there are subtle differences between them, particularly in how they handle signals such as SIGTERM and SIGKILL.

An example – building

Use the docker build command to build a container image. Give it a name and a tag.

THERE IS OFTEN A SINGLE DOT (.) AT THE END OF THE BUILD COMMAND. WHAT DOES IT CORRESPOND TO?

THERE IS OFTEN A SINGLE DOT (.) AT THE END OF THE BUILD COMMAND. WHAT DOES IT CORRESPOND TO?

- IT SETS THE BUILD CONTEXT TO THE CURRENT DIRECTORY.

IN OUR EXAMPLE, THIS MEANS THAT THE BUILD EXPECTS TO FIND THE DOCKERFILE AND THE HELLO.PY FILE IN THE DIRECTORY WHERE THE COMMAND IS INVOKED.

IF THOSE FILES AREN'T THERE, THE BUILD FAILS.

An example – the end

After the image has been built, you can run the application as a container with docker run, specifying the image name.

Do so, publishing the container's port 8000 to http://localhost:8000 on the Docker host.

HOW WOULD YOU CLARIFY THE DIFFERENCE BETWEEN THE RUN AND CMD INSTRUCTIONS?

HOW WOULD YOU CLARIFY THE DIFFERENCE BETWEEN THE RUN AND CMD INSTRUCTIONS?

- RUN EXECUTES COMMANDS DURING THE IMAGE BUILD PROCESS AND CREATES A NEW IMAGE LAYER WITH THE RESULTS. CMD SPECIFIES THE DEFAULT COMMAND TO RUN WHEN A CONTAINER STARTS FROM THE IMAGE. IT DOES NOT RUN DURING THE BUILD.

Optimizing Docker images

- Optimizing Docker images is mostly about making them smaller, faster to build, and cleaner to ship.
- Pick a sensible base image. A lot of default images come packed with tools you will never actually use in production. Switching to something lighter (like a slim or minimal variant) often cuts your image size dramatically. Just keep in mind that ultra-light images like Alpine can sometimes behave differently under the hood, so they're not always a drop-in replacement.

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

Optimizing Docker images

- Improvements can come from multi-stage builds. Think of it like separating "building" from "running." You compile or install everything in one stage, then copy only the final output into a clean image. That way, your final container doesn't carry around compilers, caches, or random build leftovers.

```
FROM node:20 AS builder

WORKDIR /app

# install dependencies first (better caching)
COPY package*.json ./
RUN npm install

# copy source code
COPY . .

# build the app
RUN npm run build


FROM node:20-alpine

WORKDIR /app

# copy only what you actually need from builder
COPY --from=builder /app/package*.json ./
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist

EXPOSE 3000

CMD ["node", "dist/index.js"]
```

Optimizing Docker images

- The order of your Dockerfile matters more than some people might expect. Docker caches each step, so if you structure it well (for example, copying dependency files first and installing them before adding your source code) you avoid reinstalling everything every time you change a single line of code. That can save time during development. It also helps to keep your layers tidy. Every instruction in a Dockerfile creates a layer, so combining related commands and cleaning up temporary files in the same step prevents unnecessary bloat. Otherwise, you end up "deleting" files in a later layer, but the space is still taken up underneath.

Optimizing Docker images

- Use .dockerignore. It is easy to accidentally send a whole project folder into the image, including things like tests, logs, .git, or environment files. A good ignore file keeps the build context lean and speeds things up without much effort.

Optimizing Docker images

- Tools like Docker BuildKit (the builder backend used by Docker) also help behind the scenes with faster builds and smarter caching.

Optimizing Docker images

- Be intentional about what actually needs to be inside the container at runtime.

  Think about runtime dependencies.

  If your application only needs a few binaries, consider copying static builds or using minimal runtimes instead of full OS images. Less stuff inside the image usually means fewer surprises later.

  Languages like Go make this especially effective since you can produce a single static binary.

Docker and air-gapped machines

Running Docker on offline (air-gapped) machines is doable, but you have to think in terms of “everything must be preloaded”.

Every file it uses already exists on disk.

Docker and air-gapped machines

Watch out when…

… pulling base images

… installing OS packages

… installing language dependencies

… downloading from URLs

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

Docker and air-gapped machines

1. Pre-pull images on an online machine, build your own images
2. Export the image(s) to a file

```
docker save -o my-images.tar myapp:1.0 nginx:latest
```

3. Transfer to offline machine
4. Load the images offline

```
docker load -i my-images.tar
```

3. Run the images

```
docker run myapp:1.0
```

Docker and air-gapped machines

If you use Compose (see later), make sure all images are preloaded, and that there is no build that requires internet dependencies (or you build everything beforehand)

Docker and air-gapped machines

`docker build` does not inherently need internet, but most Dockerfiles are written assuming internet access. Offline builds only work if everything is pre-packaged locally.

In air-gapped environments, teams usually only run `docker load` and `docker run` offline (meaning that they avoid offline builds entirely).

Docker and air-gapped machines

With our example, we have two options:

- Option 1 (very probably best): Prebuilt base image
- Option 2: Make the Dockerfile offline-safe

Docker and air-gapped machines

- Option 1 (very probably best): prebuilt base image

1. Build everything in CI (online)
2. Bake a final image
3. Export it (`docker save`)
4. Import it offline (`docker load`)
5. Never rebuild offline unless absolutely necessary

Docker and air-gapped machines

- Option 1 (very probably best): prebuilt base image

Save our example's image:

```
docker save -o flask-app.tar <name>:<tag>
```

~~Move it offline and save it offline.~~
As a replacement: Delete your image. Turn off your Wi-Fi, cut your Ethernet connection or burn your Ethernet cable if being dramatic. Try running `docker run -p 127.0.0.1:8000:8000 <name>:<tag>` *and observe that it fails.*

Run:

```
docker load -i flask-app.tar
```

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

Docker and air-gapped machines

- Option 2: make the Dockerfile offline-safe

Something like this…

Before going offline, you must collect the Ubuntu packages (download .deb files for python3 and python3-pip, using apt download or a mirror) and the Python packages on an online machine:

```
pip download flask==3.0.* -d wheels/
```

This gives you local installable .whl files.

But well…

```
FROM ubuntu:22.04

# install from pre-baked packages only (no internet)
COPY packages/ /packages/

RUN dpkg -i /packages/*.deb || true
RUN apt-get install -f -y

# install Python dependency from local wheel file
COPY wheels/ /wheels/
RUN pip install --no-index --find-links=/wheels flask==3.0.*

# install app
COPY hello.py /

ENV FLASK_APP=hello
EXPOSE 8000
CMD ["flask", "run", "--host", "0.0.0.0", "--port", "8000"]
```

Docker and air-gapped machines - installing Docker on the offline machine.

So far, everything seems pretty straightforward.

But there is one slightly annoying detail we've been glossing over: you still need to install Docker on the target machine.

In an offline environment, that can be a bit of a hassle since Docker itself and its dependencies need to be available locally before you can run any containers.

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

Docker and air-gapped machines - installing Docker on the offline machine.

To use a .tar Docker image offline, you basically "only" need Docker runtime installed that is to say:

- Docker Engine (dockerd), the daemon that manages images, containers, networks…
- Docker CLI (docker) the command-line tool you use to run commands like `docker run` and `docker ps`.

Container runtime dependencies are usually included by install. Depending on OS, Docker should bring containerd, runc, networking setup. You don't manually manage these in most cases.

Also, not strictly required, but often needed in real environments are:

- ca-certificates (for TLS-enabled registries, sometimes still useful offline)
- iptables support (networking)
- kernel compatibility (Linux features like cgroups, namespaces)

Docker and air-gapped machines - installing Docker on the offline machine.

- On Linux, it should be straightforward. If you download the required packages in advance (.deb on Ubuntu/Debian, .rpm on RHEL/Fedora), you can transfer them to the offline machine and install them locally.
- On Windows, remember the 2 teams: Docker Desktop vs working directly in WSL 2.

Docker and air-gapped machines - installing Docker on the offline machine.

In the end, it should look like this (*easy* Linux setup):

Part 1 – online machine

```
mkdir docker-offline && cd docker-offline
```

```
sudo apt-get update
```

```
apt-get download docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

```
sudo apt-get install --download-only docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Docker and air-gapped machines - installing Docker on the offline machine.

In the end, it should look like this (*easy* Linux setup):

Transfer files to offline machine

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

Docker and air-gapped machines - installing Docker on the offline machine.

In the end, it should look like this (*easy* Linux setup):

Part 1 – offline machine

```
cd ~/docker-offline
```

```
sudo dpkg -i *.deb
```

```
sudo apt-get install -f
```

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

And then load image.tar

Defining services, networks, and volumes (data storage and persistence)

Services

A service is just a container with a defined role. In Docker Compose (see later), a service usually means “one part of the application that runs continuously”.

```
services:
  web:
    image:
myapp:1.0
    ports:
      - "8000:8000"
```

web = service name

Runs the app container

Defines how it starts, what ports it exposes

Defining services, networks, and volumes (data storage and persistence)

Networks

A **network** lets containers communicate with each other: “internal private LAN for containers”

By default, Docker creates a network for you, but you can define your own:

```
networks:
  app-net:
```

You can then attach services:

```
services:
  web:
    networks:
      - app-net

  db:
    networks:
      - app-net
```

web can reach db using its service name.

No need for IP addresses.

Defining services, networks, and volumes (data storage and persistence)

Networks

Previously we used Docker Compose syntax to introduce the concept.

Docker provides CLI commands to create networks manually…

```
docker network create app-net
```

…while Docker Compose can create them automatically.

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

Defining services, networks, and volumes (data storage and persistence)

Volumes

A volume is how Docker keeps data even if containers are deleted.

- Without volumes: container data is lost when it stops
- With volumes: data survives restarts and rebuilds

```
services:
  db:
    image: postgres
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```

/var/lib/postgresql/data is where Postgres stores its data

db-data is a persistent storage managed by Docker

Even in the case of `docker rm -f db`, the database data is still there.

Defining services, networks, and volumes (data storage and persistence)

Volumes

Docker CLI syntax:

```
docker volume create db-data
```

BUILDING CUSTOM IMAGES AND DOCKER COMPOSE

Defining services, networks, and volumes (data storage and persistence)

```
services:
  web:
    image: myapp
    ports:
      - "8000:8000"
    networks:
      - app-net

  db:
    image: postgres
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-net

networks:
  app-net:

volumes:
  db-data:
```

Docker Compose

Docker Compose is a tool that lets you define and run multi-container applications using a single file instead of a bunch of separate docker run commands. The idea is to have a declarative file that makes everything reproducible and easy to share.

Instead of doing this manually:

```
docker run myapp
docker run postgres
docker network create app-net
docker volume create db-data
```

You write everything once in a file and run:

```
docker compose up
```

Docker Compose – the Compose file (docker-compose.yml)

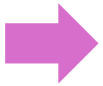
This is where we define our app setup, thanks to the concepts we just introduced!

```
services:
  web:
    image: myapp
    ports:
      - "8000:8000"
    networks:
      - app-net

  db:
    image: postgres
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-net

networks:
  app-net:

volumes:
  db-data:
```



This is our Compose file!

Docker Compose – the Compose file (docker-compose.yml)

Common commands

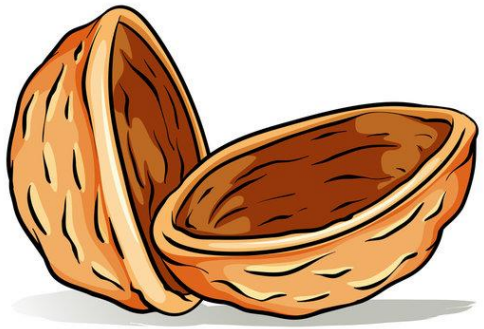
Start everything: `docker compose up`

Run in background: `docker compose up -d`

Stop everything: `docker compose down`

Remove volumes too (careful with data): `docker compose down -v`

Docker Compose



Docker Compose exists because real apps usually have a frontend, a backend, a database, maybe a cache (Redis…).

Managing all that with docker run gets messy fast.

Compose turns that into “one file, one command, full system”.

WHAT IS THE PRIMARY PURPOSE OF A DOCKER-COMPOSE.YML FILE?

- ❑ BUILD DOCKER IMAGES ONLY
- ❑ DEFINE AND RUN MULTI-CONTAINER APPLICATIONS
- ❑ CREATE VIRTUAL MACHINES
- ❑ STORE DOCKER LOGS

WHAT IS THE PRIMARY PURPOSE OF A DOCKER-COMPOSE.YML FILE?

- ❑ BUILD DOCKER IMAGES ONLY
- ☑ DEFINE AND RUN MULTI-CONTAINER APPLICATIONS
- ❑ CREATE VIRTUAL MACHINES
- ❑ STORE DOCKER LOGS

```
services:
  db:
    image:
postgres
```

IN THIS COMPOSE FILE, WHAT IS DB?

- ❑ AN IMAGE
- ❑ A CONTAINER
- ❑ A SERVICE
- ❑ A NETWORK
- ❑ A VOLUME

```
services:
  db:
    image:
postgres
```

IN THIS COMPOSE FILE, WHAT IS DB?

- ☐ AN IMAGE
- ☐ A CONTAINER
- ☑ A SERVICE
- ☐ A NETWORK
- ☐ A VOLUME

WHICH COMMAND STARTS ALL SERVICES DEFINED IN A COMPOSE FILE?

- ❑ DOCKER START
- ❑ DOCKER RUN
- ❑ DOCKER COMPOSE UP
- ❑ DOCKER SERVICE CREATE

WHICH COMMAND STARTS ALL SERVICES DEFINED IN A COMPOSE FILE?

- ☐ DOCKER START
- ☐ DOCKER RUN
- ☑ DOCKER COMPOSE UP
- ☐ DOCKER SERVICE CREATE

WHICH RESOURCES ARE USUALLY CREATED AUTOMATICALLY BY DOCKER COMPOSE?

- ❑ CONTAINERS
- ❑ IMAGES
- ❑ NETWORKS
- ❑ NAMES VOLUMES

WHICH RESOURCES ARE USUALLY CREATED AUTOMATICALLY BY DOCKER COMPOSE?

- [x] CONTAINERS
- [ ] IMAGES
- [x] NETWORKS
- [x] NAMES VOLUMES

WHAT IS THE DIFFERENCE BETWEEN IMAGE AND BUILD?

```
services:
  app:
    image:
myapp
```

```
services:
  app:
    build: .
```

```
services:
  app:
    image:
myapp
```

```
services:
  app:
    build: .
```

WHAT IS THE DIFFERENCE BETWEEN IMAGE AND BUILD?

- IMAGE PULLS/USES AN IMAGE, BUILD BUILDS ONE FROM A DOCKERFILE

A DOCKER COMPOSE SERVICE CAN USE:

- ❑ AN IMAGE
- ❑ VOLUMES
- ❑ NETWORKS
- ❑ ENVIRONMENT VARIABLES

A DOCKER COMPOSE SERVICE CAN USE:

- ☑ AN IMAGE
- ☑ VOLUMES
- ☑ NETWORKS
- ☑ ENVIRONMENT VARIABLES